Отчёт по лабораторной работе №4

Установка и конфигурирование дополнительного ПО и инструментов разработчика в Fedora 41

Кулаженкова Яна Сергеевна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Установка дополнительных репозиториев и пакетов 8
4.2 Настройка pnpm и установка глобальных пакетов 10
4.3 Создание и настройка Git-репозитория 11
4.4 Конфигурация package.json и настройка commitizen 12
4.5 Инициализация Git Flow 12
4.6 Работа с ветками и создание первого релиза 13
4.7 Генерация CHANGELOG и завершение релиза 14
4.8 Создание релиза на GitHub 15
4.9 Работа с feature-ветками 16
4.10 Обновление версии и создание нового релиза 17

5 Выводы **19**

6 Список литературы **20**

Список иллюстраций

4.1 Подключение репозитория COPR и установка gitflow 8
4.2 Импорт GPG-ключа репозитория 9
4.3 Установка nodejs и pnpm 9
4.4 Настройка pnpm и установка commitizen 10
4.5 Установка standard-changelog 10
4.6 Инициализация Git-репозитория 11
4.7 Подключение удалённого репозитория и отправка изменений 11
4.8 Создание и редактирование package.json 12
4.9 Инициализация Git Flow 13
4.10 Отправка веток в удалённый репозиторий 13
4.11 Создание релизной ветки 14
4.12 Генерация CHANGELOG и завершение релиза 15
4.13 Отправка изменений и тегов в удалённый репозиторий 15
4.14 Создание релиза на GitHub через GitHub CLI 16
4.15 Работа с feature-веткой 16
4.16 Редактирование package.json для нового релиза 17
4.17 Завершение второго релиза 18
4.18 Отправка второго релиза и создание релиза на GitHub 18

Список таблиц

1 Цель работы

Целью данной работы является получение навыков установки и настройки инструментов разработчика в среде Fedora 41, включая пакетный менеджер pnpm, инструменты для работы с Git (gitflow, commitizen, standard-changelog) и освоение работы с GitHub CLI.

2 Задание

1. Установить и настроить репозиторий COPR для установки gitflow.
2. Установить инструменты разработчика: nodejs, pnpm, gitflow.
3. Настроить pnpm и установить глобальные пакеты commitizen и standard-changelog.
4. Создать и настроить Git-репозиторий с поддержкой Git Flow.
5. Настроить commitizen для создания структурированных коммитов.
6. Сгенерировать CHANGELOG с помощью standard-changelog.
7. Создать релизы на GitHub с использованием GitHub CLI.
8. Освоить работу с feature-ветками в Git Flow.

3 Теоретическое введение

COPR (Cool Other Packages Repo) — система сборки пакетов для Fedora, позволяющая пользователям создавать и распространять пакеты, не входящие в официальные репозитории.

pnpm — быстрый и эффективный менеджер пакетов для JavaScript/Node.js, альтернатива npm и yarn.

Git Flow — набор расширений для Git, реализующих модель ветвления для управления релизами.

Commitizen — инструмент для создания стандартизированных сообщений коммитов в соответствии с Conventional Commits.

standard-changelog — утилита для автоматической генерации CHANGELOG.md на основе коммитов, следующих соглашению Conventional Commits.

GitHub CLI — официальный инструмент командной строки для взаимодействия с GitHub.

4 Выполнение лабораторной работы

4.1 Установка дополнительных репозиториев и пакетов

Первым этапом работы стала установка gitflow из репозитория COPR. Для этого был подключён репозиторий `elegans/gitflow` с помощью команды `sudo dnf copr enable elegans/gitflow`, после чего выполнена установка пакета `gitflow` (рис. [**fig:004**]).

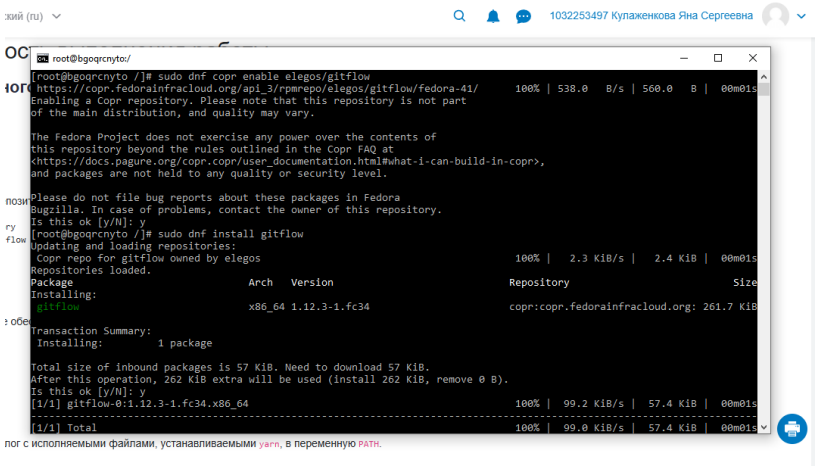

Рисунок 4.1: Подключение репозитория COPR и установка gitflow

В процессе установки произошёл импорт GPG-ключа для подтверждения подлинности пакетов из подключённого репозитория (рис. [**fig:041**]).

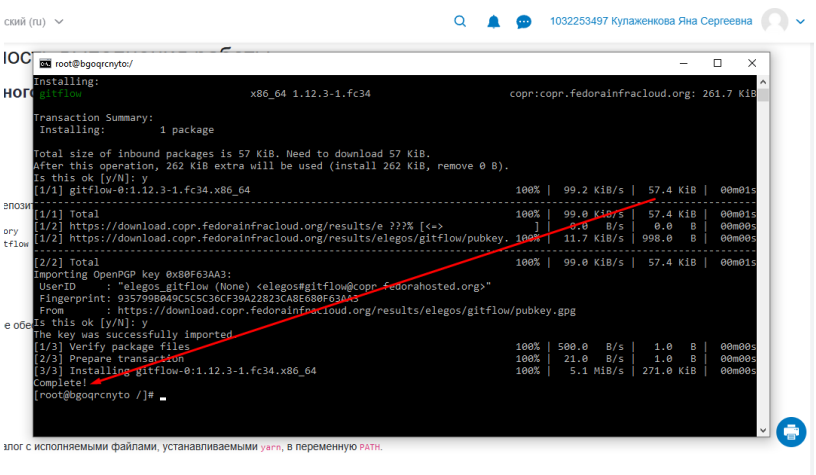

Рисунок 4.2: Импорт GPG-ключа репозитория

Далее была выполнена установка пакетов `nodejs` и `pnpm`. Вместе с ними были установлены зависимости: `nodejs-libs`, `nodejs-docs`, `nodejs-full-i18n` и `nodejs-npm`. Общий размер устанавливаемых пакетов составил 231 МБ (рис. [**fig:042**]).

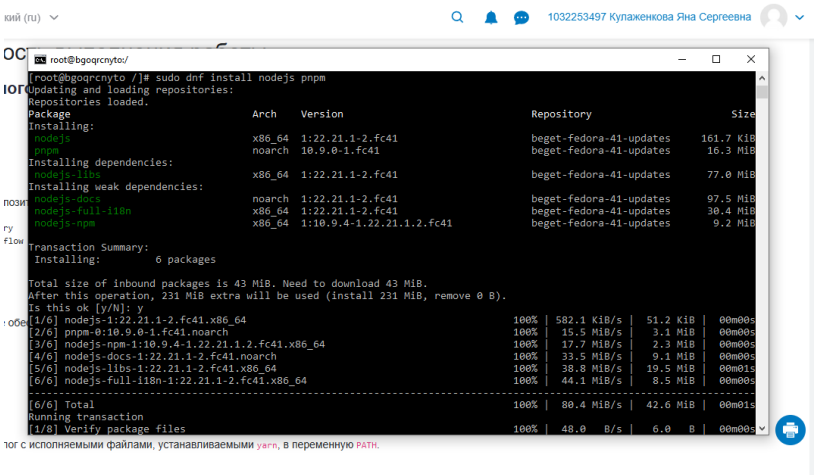

Рисунок 4.3: Установка nodejs и pnpm

4.2 Настройка pnpm и установка глобальных пакетов

После установки pnpm была выполнена его первоначальная настройка с помощью команды `pnpm setup`, которая добавила необходимые переменные окружения в файл `.bashrc`. После применения изменений через `source ~/.bashrc` были установлены глобальные пакеты `commitizen` (рис. [**fig:043**]) и `standard-changelog` (рис. [**fig:044**]).

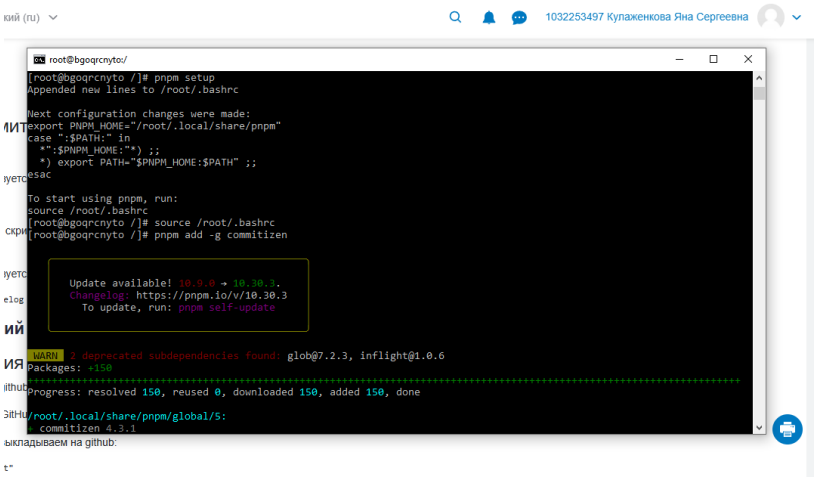

Рисунок 4.4: Настройка pnpm и установка commitizen

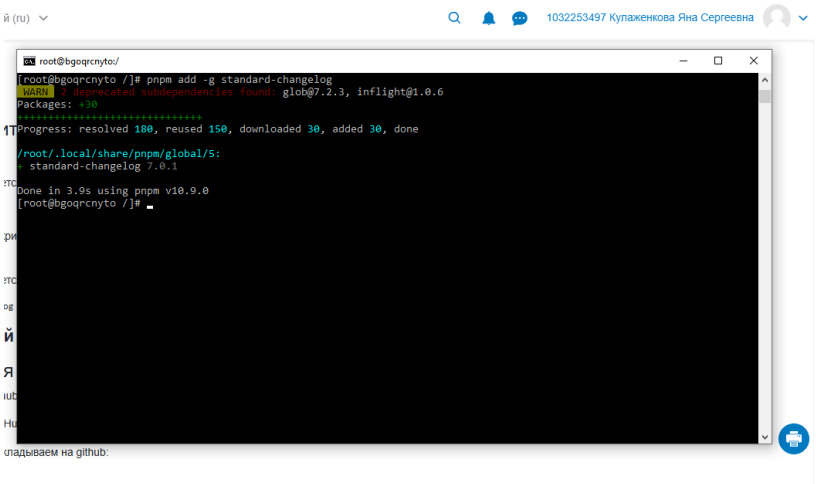

Рисунок 4.5: Установка standard-changelog

4.3 Создание и настройка Git-репозитория

Для выполнения дальнейших работ был создан каталог `~/projects/git-extended`, инициализирован Git-репозиторий и выполнен первый коммит с файлом README.md (рис. [**fig:045**]).

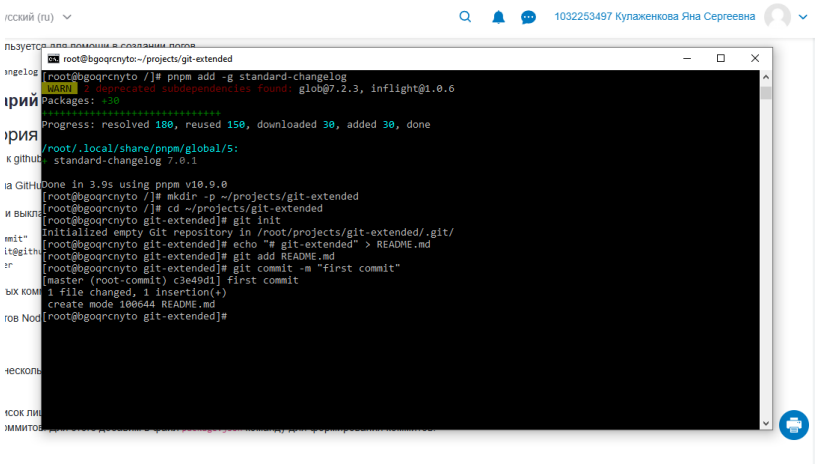

Рисунок 4.6: Инициализация Git-репозитория

Репозиторий был связан с удалённым репозиторием на GitHub (`git@github.com:Yana-nka/git-extended.git`), после чего выполнена отправка изменений. При первом подключении к GitHub было подтверждение подлинности хоста (рис. [**fig:046**]).

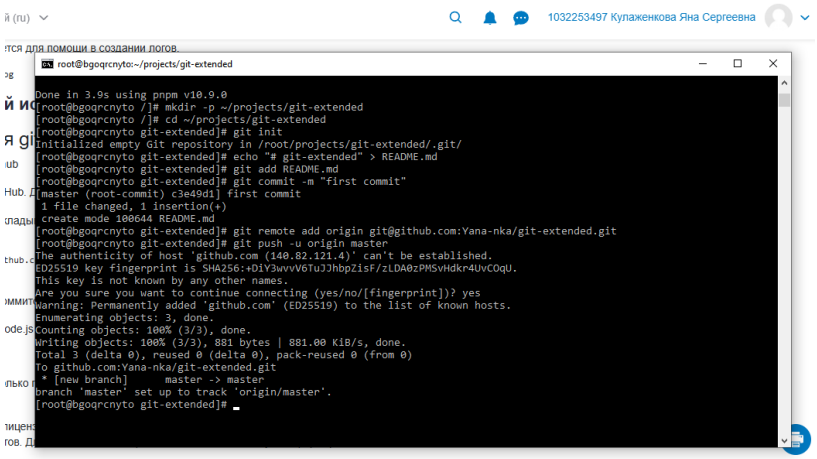

Рисунок 4.7: Подключение удалённого репозитория и отправка изменений

4.4 Конфигурация package.json и настройка commitizen

В корне репозитория был создан файл `package.json` с базовой конфигурацией проекта, включая указание менеджера пакетов `pnpm@10.9.0` (рис. [**fig:047**]).

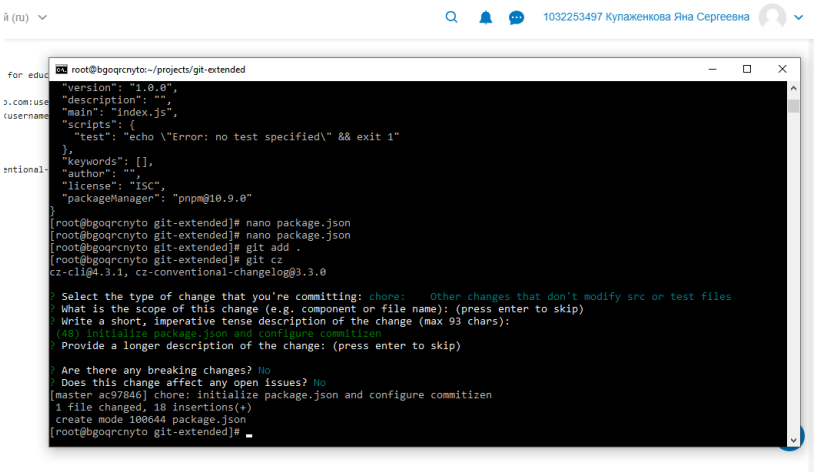

Рисунок 4.8: Создание и редактирование package.json

С помощью команды `git cz` был выполнен первый структурированный коммит, соответствующий стандарту Conventional Commits. Тип коммита выбран `chore`, описание — «initialize package.json and configure commitizen».

4.5 Инициализация Git Flow

В репозитории была выполнена инициализация Git Flow. Поскольку репозиторий уже был инициализирован ранее, использована команда с флагом принудительной инициализации `git flow init -f`. В процессе были настроены названия веток: - Ветка для релизов: `master` - Ветка для разработки: `develop` - Префиксы для вспомогательных веток: `feature/`, `bugfix/`, `release/`, `hotfix/` - Префикс для тегов: `v`

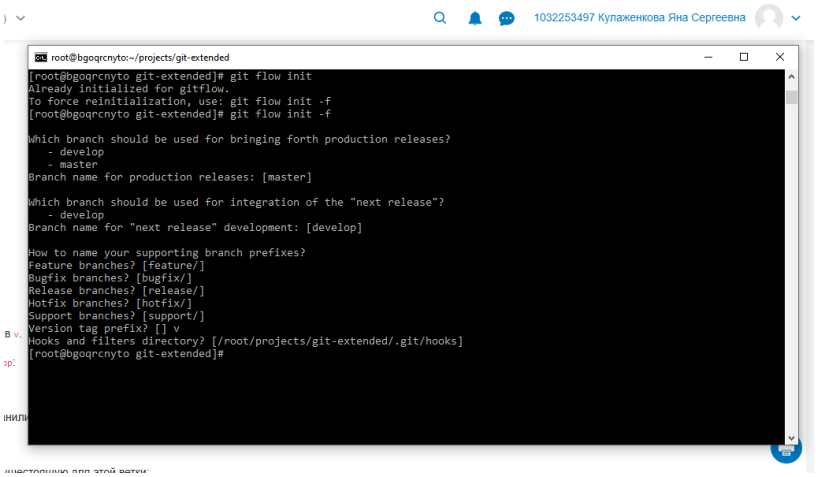

Рисунок 4.9: Инициализация Git Flow

4.6 Работа с ветками и создание первого релиза

После инициализации в репозитории присутствуют две основные ветки: `develop` и `master`. Обе ветки были отправлены в удалённый репозиторий, при этом для ветки `develop` настроено отслеживание удалённой ветки (рис. [**fig:049**]).

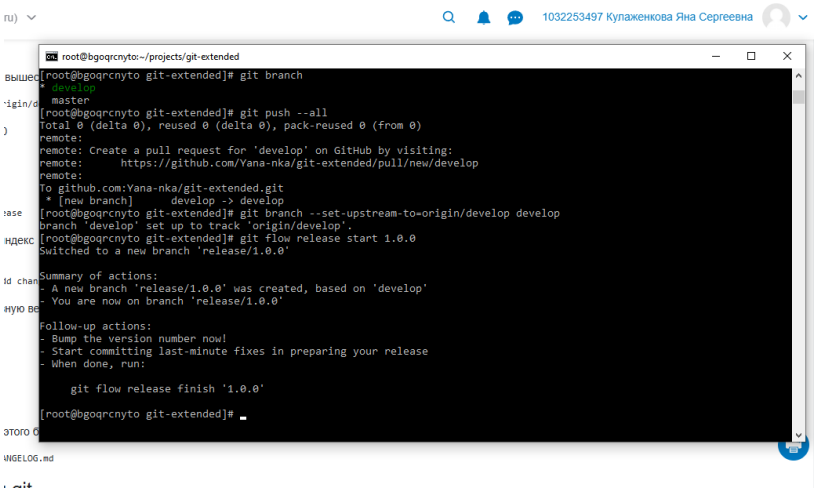

Рисунок 4.10: Отправка веток в удалённый репозиторий

Был создан первый релиз с помощью команды `git flow release start`

`1.0.0`, создана ветка `release/1.0.0` на основе ветки `develop` (рис. [**fig:050**]).

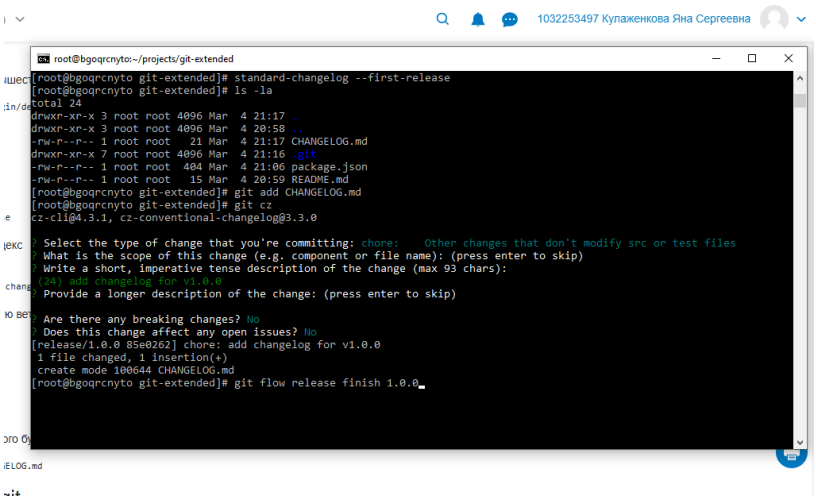

Рисунок 4.11: Создание релизной ветки

4.7 Генерация CHANGELOG и завершение релиза

С помощью утилиты `standard-changelog --first-release` был сгенерирован файл `CHANGELOG.md`. Сгенерированные изменения были закоммичены с помощью `git cz` с типом `chore` и описанием «add changelog for v1.0.0». После этого выполнено завершение релиза `git flow release finish 1.0.0` (рис. [**fig:051**]).

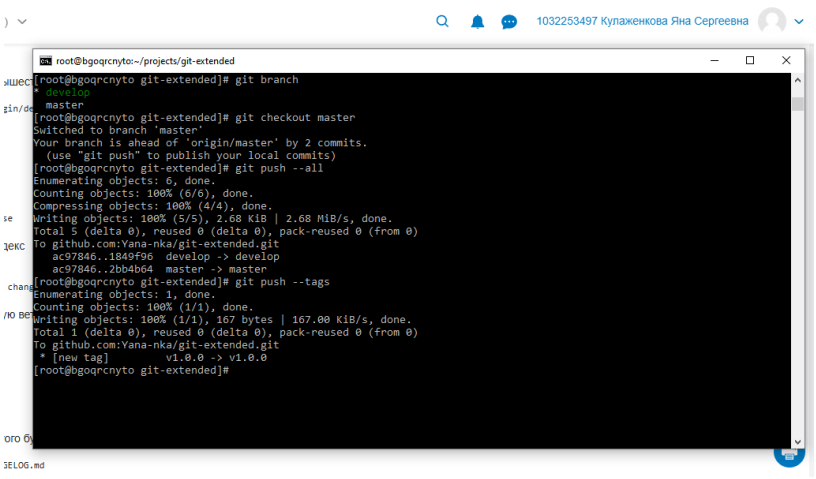

Рисунок 4.12: Генерация CHANGELOG и завершение релиза

После завершения релиза изменения были отправлены в удалённый репозиторий (ветки `master` и `develop`), а также создан тег `v1.0.0` (рис. [**fig:052**]).

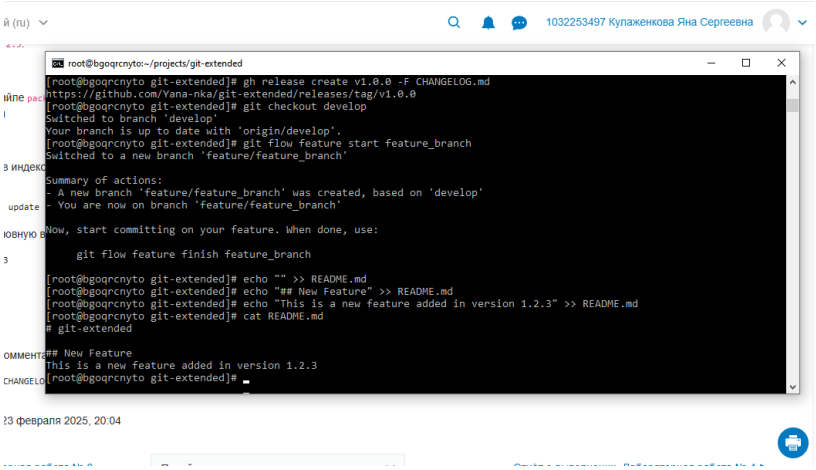

Рисунок 4.13: Отправка изменений и тегов в удалённый репозиторий

4.8 Создание релиза на GitHub

С помощью GitHub CLI был создан релиз на основе тега `v1.0.0` с описанием из файла `CHANGELOG.md` (рис. [**fig:053**]).

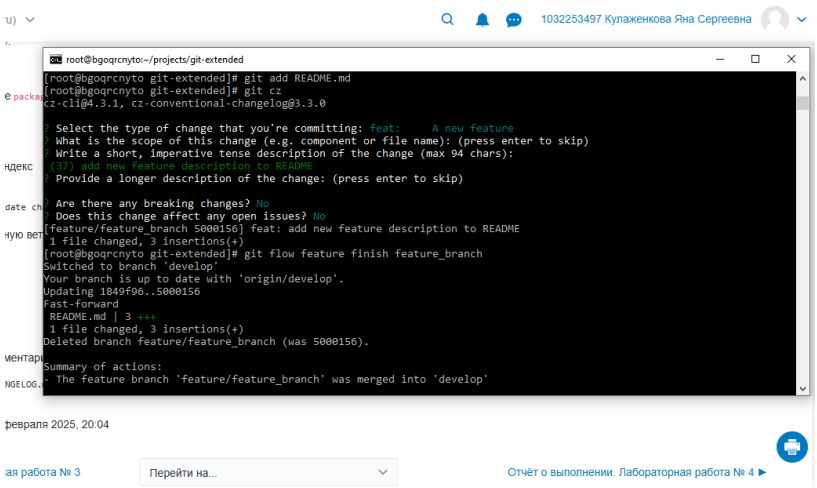

Рисунок 4.14: Создание релиза на GitHub через GitHub CLI

4.9 Работа с feature-ветками

Для добавления новой функциональности была создана feature-ветка `feature/feature_branch`. В ней в файл `README.md` были добавлены строки с описанием новой функции. Изменения были закоммичены с типом `feat` (новая функциональность) и описанием «add new feature description to README» (рис. [**fig:054**]).

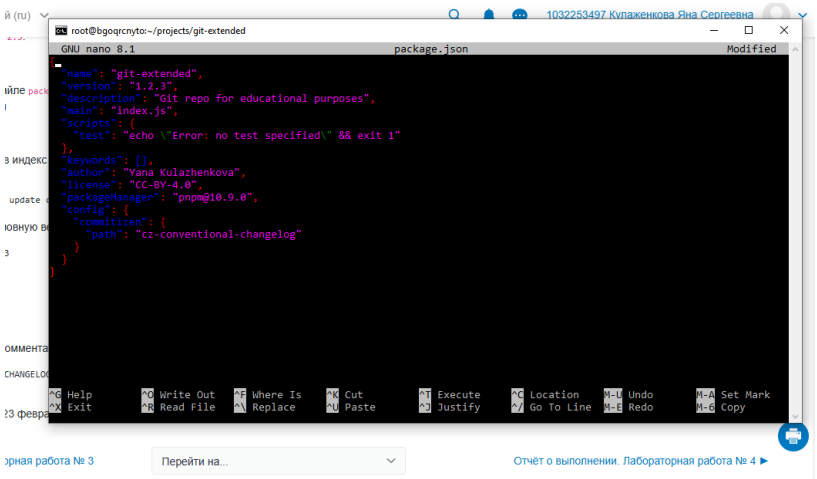

Рисунок 4.15: Работа с feature-веткой

После завершения работы feature-ветка была объединена с `develop` с помощью `git flow feature finish feature_branch`.

4.10 Обновление версии и создание нового релиза

Для создания нового релиза была изменена версия в файле `package.json` на `1.2.3`, а также обновлены поля `author` и `license` (рис. [**fig:055**]).

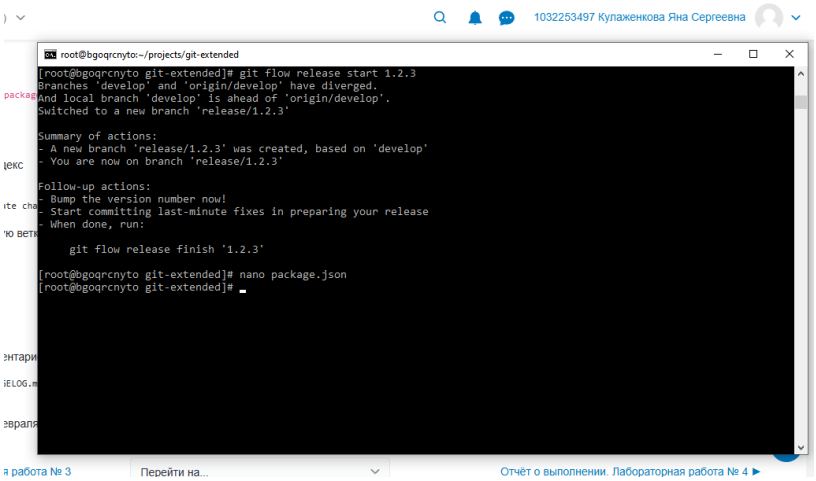

Рисунок 4.16: Редактирование package.json для нового релиза

Был создан новый релиз `git flow release start 1.2.3`. После обновления версии и повторной генерации CHANGELOG изменения закоммичены с типом `chore` и завершён релиз (рис. [**fig:056**]).

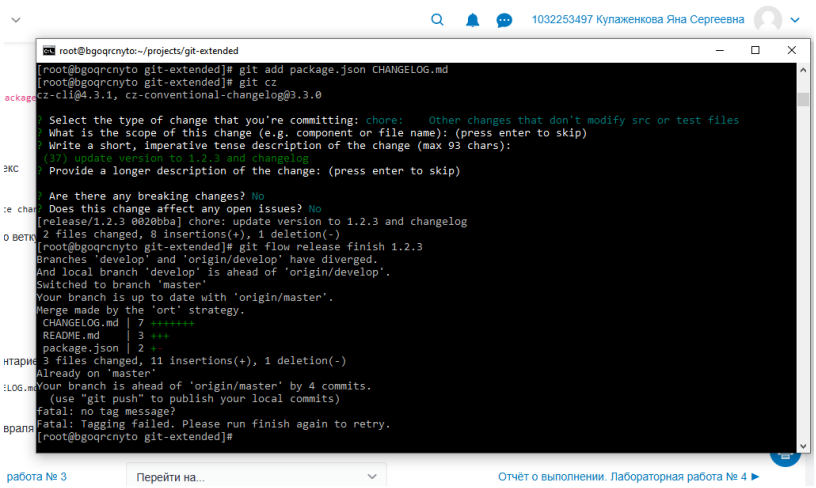

Рисунок 4.17: Завершение второго релиза

После завершения релиза все изменения отправлены в удалённый репозиторий, создан тег `v1.2.3` и соответствующий релиз на GitHub (рис. [**fig:057**]).

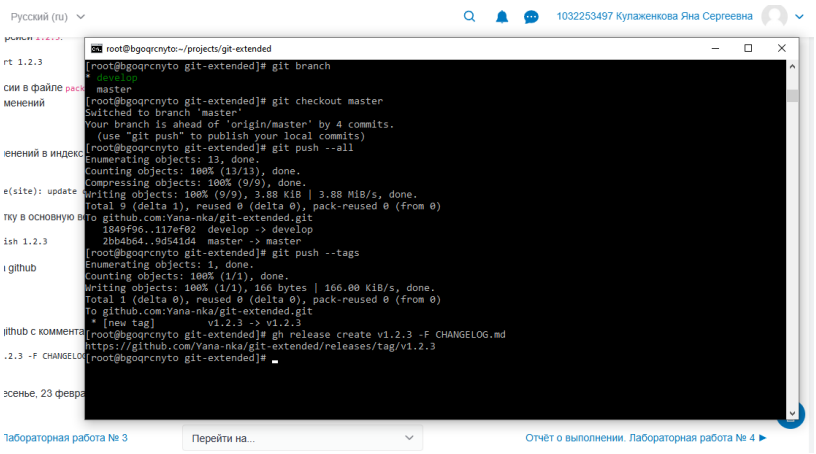

Рисунок 4.18: Отправка второго релиза и создание релиза на GitHub

5 Выводы

В ходе выполнения лабораторной работы были успешно установлены и настроены инструменты разработчика в среде Fedora 41:

- Подключён репозиторий COPR и установлен gitflow.
- Установлены nodejs и pnpm, выполнена настройка окружения.
- Установлены и настроены commitizen и standard-changelog.
- Освоена работа с Git Flow: создание релизных и feature-веток.
- Настроена генерация CHANGELOG на основе структурированных коммитов.
- Освоена работа с GitHub CLI для создания релизов.
- Выполнено два полных цикла разработки с созданием релизов версий 1.0.0 и 1.2.3.

Полученные навыки являются основой для организации структурированной разработки программных проектов с использованием современных инструментов и методологий.

6 Список литературы

1. Официальная документация Git Flow. URL: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
2. Документация Commitizen. URL: https://commitizen-tools.github.io/commitizen/
3. Conventional Commits. URL: https://www.conventionalcommits.org/
4. Документация pnpm. URL: https://pnpm.io/ru/
5. GitHub CLI Manual. URL: https://cli.github.com/manual/